

FinSight

Complete Learning Guide

Educational documentation for understanding and rebuilding a Retrieval-Augmented Generation (RAG) system for financial documents.

Topics: architecture, tools, data flow, code organization, workflows, challenges, best practices, and step-by-step rebuild guide.

Table of Contents

FinSight — Complete Learning Guide	1
1. Introduction	1
2. Core Concepts (Beginner Primer)	2
3. High-Level Architecture	3
4. Technology Stack	4
5. Project Structure	4
6. Configuration	4
7. Data Models	5
8. Step-by-Step Development Journey	5
9. Ingestion Pipeline (Deep Dive)	6
10. Query Pipeline (Deep Dive)	7
11. Prompt Engineering	8
12. API Layer	8
13. Frontend (Streamlit)	9
14. Integration Between Components	9
15. Challenges and Solutions	10
16. Best Practices Followed	10
17. Testing Strategy	11
18. How to Run the Full System	11
19. Rebuilding From Scratch (Checklist)	12
20. Glossary	12
21. Further Learning	12
22. Appendix A — Environment Variable Reference	12
23. Appendix B — Code Walkthrough: Chunk ID Generation	13
24. Appendix C — Code Walkthrough: ChromaStore Query	13
25. Appendix D — API Request/Response Examples	14
26. Appendix E — Ingestion Module Details	14
27. Appendix F — LLM Provider Details	15
28. Appendix G — Frontend Session Flow	15
29. Appendix H — Decision Log	16
30. Appendix I — Troubleshooting Guide	16

FinSight — Complete Learning Guide

Educational documentation for understanding and rebuilding this project from scratch.

1. Introduction

1.1 What is FinSight?

FinSight is a Retrieval-Augmented Generation (RAG) application built for financial documents. It lets you upload PDFs such as annual reports, earnings call transcripts, investor presentations, and SEBI filings, then ask questions in plain English and receive answers grounded in the document text, with page-level citations.

RAG means the system does not rely on the LLM's memory alone. Instead it:

- Retrieves relevant passages from your indexed documents.
- Augments the LLM prompt with those passages as context.
- Generates an answer that should cite only what was retrieved.

This reduces hallucination compared to asking ChatGPT a question with no document access.

1.2 Project Objectives

Objective	How FinSight achieves it
Ingest PDF financial reports	PyMuPDF loader + cleaning + chunking pipeline
Search by meaning, not keywords	Vector embeddings + ChromaDB similarity search...
Answer with citations	Source objects carry company, file, page, sco...
Compare two companies	Per-company retrieval + comparison prompt
Run locally without API costs	Default: local BGE embeddings + Ollama LLM
Optional cloud AI	Switch to Gemini via environment variables
Usable by non-developers	Streamlit chat UI + FastAPI backend

1.3 Who This Guide Is For

This document assumes basic Python familiarity (functions, classes, virtual environments) but does not assume prior knowledge of RAG, vector databases, or LLMs. Every major concept is explained before it is used in the code.

1.4 What This Guide Covers

Only tracked project source code is described — the implementation you can see in the repository. Local runtime artifacts (uploaded PDFs, Chroma database files, .env secrets) are intentionally excluded from version control via .gitignore and are not part of this guide's file walkthrough.

2. Core Concepts (Beginner Primer)

2.1 Large Language Models (LLMs)

An LLM is a neural network trained to predict text. Given a prompt, it generates a continuation. Examples used in FinSight:

- Ollama + llama3 — runs on your machine, free, private.
- Gemini 2.0 Flash — Google's cloud API, requires an API key.

The LLM in FinSight never reads entire PDFs at query time. It only sees small retrieved chunks plus your question.

2.2 Embeddings

An embedding converts text into a list of numbers (a vector). Similar meanings produce vectors that are close together in mathematical space.

FinSight default: BAAI/bge-small-en-v1.5 (384 dimensions) via sentence-transformers.

- Document chunks are embedded when ingesting.
- User questions are embedded at query time.
- ChromaDB finds chunks whose vectors are closest to the question vector.

2.3 Vector Database (ChromaDB)

A vector database stores embeddings and supports similarity search: "give me the k vectors closest to this query

vector."

FinSight uses one persistent collection named `finsight_docs`. Metadata filters (`company`, `year`, `doc_type`) narrow results without separate collections per company.

2.4 Chunking

PDFs are too long to embed as one piece. FinSight splits text into chunks (~1000 characters with 150 overlap) using LangChain's `RecursiveCharacterTextSplitter`. Overlap helps avoid cutting sentences mid-thought at chunk boundaries.

2.5 Metadata

Each chunk carries metadata:

- `company` (e.g. TCS, IOC)
- `doc_type` (annual_report, transcript, presentation, filing)
- `page` (1-indexed page number)
- `source_file` (PDF filename)
- `chunk_id` (deterministic hash — same text → same id)
- optional `year`, `quarter`

Metadata enables filtering ("only TCS") and citations ("page 115").

3. High-Level Architecture

3.1 System Diagram



3.2 Request Flow Summary

Ingest: PDF file → text pages → cleaned text → chunks → embedding vectors → stored in Chroma.

Query: Question → query embedding → top-k similar chunks → prompt assembly → LLM answer + source list.

Comparison query: Same as query, but retrieval runs separately per company, then a specialized comparison prompt is used.

4. Technology Stack

Layer	Technology	Purpose	Why chosen
Language	Python 3.11+	Entire stack	Rich ML/API ecosystem
Config	Pydantic Settings	Type-safe <code>`.env`</code> loading	Validation + defaults
API	FastAPI	REST endpoints	Async-friendly, auto OpenAPI docs
Server	Uvicorn	ASGI server	Standard for FastAPI
PDF	PyMuPDF ('fitz')	Extract text per page	Fast, reliable text PDFs
Chunking	LangChain text splitters	Recursive character split	Battle-tested defaults
Embeddings (default)	sentence-transformers + BGE-sma	Local vectors	No API limits, free
Embeddings (alt)	Google Gemini embedding API	Cloud vectors	When local GPU/RAM limited
Vector DB	ChromaDB 0.5.x	Persistent similarity search	Simple local setup
LLM (default)	Ollama + llama3	Local inference	Free, private
LLM (alt)	Gemini 2.0 Flash	Cloud generation	Faster if API available
Frontend	Streamlit	Chat + sidebar UI	Rapid prototyping
HTTP client	<code>`requests`</code>	Frontend → API	Simple, sufficient
Testing	pytest	Unit + API tests	Standard Python testing
Linting	Ruff	Style + imports	Fast linter

5. Project Structure

```
fin-sight/
├── config/settings.py    # All environment-driven settings
├── api/                  # FastAPI REST layer
│   ├── main.py          # Routes and thread-pool wrappers
│   ├── deps.py          # Shared dependencies (Chroma singleton)
│   └── schemas.py       # Pydantic request/response models
├── src/fin-sight/       # Core business logic (importable library)
│   ├── models.py        # Chunk, Source, QueryResponse, IngestResult
│   ├── errors.py        # GeminiQuotaError
│   ├── ingestion/       # PDF → chunks
│   ├── embeddings/      # Local + Gemini providers
│   ├── vectorstore/     # ChromaDB wrapper
│   ├── retrieval/       # Search + comparison logic
│   ├── llm/             # Ollama + Gemini providers
│   ├── prompts/         # System prompts + context builders
│   └── pipeline/        # ingest_document, answer_question
├── frontend/           # Streamlit UI
├── scripts/            # CLI ingest and query
├── tests/              # pytest suite (56 tests)
├── docs/               # This learning guide
└── data/documents/.gitkeep # PDF upload location (runtime)
```

```
└─ storage/chroma/.gitkeep # Vector DB location (runtime)
```

Design principle: src/finsight has no HTTP dependencies. The API, CLI, and tests all call the same pipeline functions. This avoids duplicating business logic.

6. Configuration

File: config/settings.py

Settings load from environment variables and .env (copy from .env.example). Key defaults:

```
LLM_PROVIDER=ollama
OLLAMA_MODEL=llama3:latest
EMBEDDING_PROVIDER=local
LOCAL_EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
CHUNK_SIZE=1000
CHUNK_OVERLAP=150
RETRIEVAL_TOP_K=5
COMPARISON_K_PER_COMPANY=4
```

get_settings() is cached with @lru_cache so settings are read once per process.

Provider factories (get_embedder(), get_llm()) are also cached — the BGE model and LLM clients load once, not on every request. This was a critical performance fix.

7. Data Models

File: src/finsight/models.py

ChunkMetadata + Chunk

A Chunk is the atomic unit stored in Chroma:

- text — the passage content
- metadata — company, page, source_file, chunk_id, etc.

chunk_id is a SHA-1 hash of (source_file, page, text prefix). Re-ingesting the same content overwrites the same id (upsert), preventing duplicates.

Source

Returned at query time. Includes score (similarity, higher = better) for ranking and display.

QueryResponse

```
QueryResponse(answer="...", sources=[Source(...), ...])
```

IngestResult

Reports chunks_added, chunks_skipped, partial (True if stopped mid-run due to rate limit).

8. Step-by-Step Development Journey

FinSight was built in nine incremental steps. Understanding this order helps you rebuild it yourself.

Step 1 — Project Skeleton + Config

- Created folder layout, requirements.txt, Pydantic settings.
- Decision: Single .env file at project root; paths resolved relative to ROOT_DIR.

Step 2 — Ingestion (Loader, Cleaner, Chunker)

- Loader (ingestion/loader.py): PyMuPDF extracts (page_number, text) per page.
 - Cleaner (ingestion/cleaner.py): Removes repeated headers/footers and page numbers that would pollute embeddings.
- Chunker (ingestion/chunker.py): LangChain splitter produces Chunk objects with metadata.

Step 3 — Embeddings Provider

- Abstract EmbeddingProvider base class.
- LocalEmbeddings — BGE model with query prefix for retrieval.
- GeminiEmbeddings — API with batching, retries, quota detection.

Step 4 — Vector Store

- ChromaStore wraps persistent Chroma with cosine distance.
- Explicit embeddings passed in (Chroma does not call external APIs itself).

Step 5 — Ingest Pipeline

- Orchestrates load → clean → chunk → embed → store.
- Batch persistence: each embedding batch is written immediately.
- Resume support: skip chunk_ids already in DB.

Step 6 — LLM + Prompts

- OllamaLLM and GeminiLLM implement generate(system, user).
- Prompt templates enforce: cite sources, use only context, Indian FY / crore rules for comparisons.

Step 7 — Retrieval + Query Pipeline

- Standard retrieve() — one embedding search.
- Comparison mode — retrieve_per_company() with financial query boosting.
- answer_question() ties retrieval, prompts, and LLM together.

Step 8 — FastAPI Backend

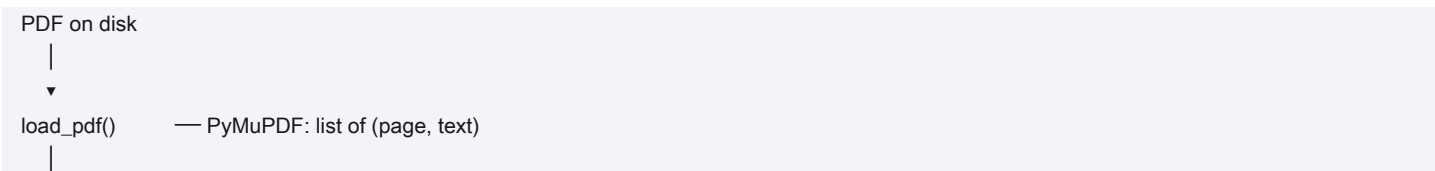
- REST endpoints for upload, ingest, query, document management.
- Blocking work runs in run_in_threadpool so /health stays responsive.

Step 9 — Streamlit Frontend

- Sidebar: upload, ingest, document list, query mode.
- Main area: chat history with expandable citations.

9. Ingestion Pipeline (Deep Dive)

9.1 Flowchart



```

▼
clean_pages() — Remove boilerplate lines
|
▼
chunk_pages() — RecursiveCharacterTextSplitter
|
▼
[optional] delete_by_source() if replace_existing=True
|
▼
existing_chunk_ids() if resume=True
|
▼
For each batch of N chunks:
    embed_documents(batch texts)
    store.add_chunks(batch, vectors)
    [sleep batch_delay for Gemini rate limits]
    |
    ▼
IngestResult

```

9.2 Key Function

File: `src/finsight/pipeline/ingest_pipeline.py` — `ingest_document()`

Parameters that matter:

- `replace_existing=True` — wipe old chunks for this PDF first (clean re-ingest).
- `resume=True` — skip chunks already stored (continue after rate limit).
- `company, doc_type, year` — stored in metadata for filtering.

9.3 Example CLI

```

source .venv/bin/activate
python scripts/ingest.py data/documents/annual-report-2025-2026.pdf \
--company TCS --doc-type annual_report --year 2026 --replace

```

10. Query Pipeline (Deep Dive)

10.1 Standard Query Flow

```

User question
|
▼
store.count() == 0? —yes→ friendly "no documents" message (no LLM call)
|
no
▼
_should_compare()? —no→ retrieve() with optional company filter
|           |
yes         ▼
            build_rag_prompt()
retrieve_per_company() |
|                       ▼
|                       llm.generate()
build_comparison_prompt() |
|                           ▼
└─────────────────────────→ QueryResponse

```


10.2 Empty Store Guard

File: `src/finsight/pipeline/query_pipeline.py`

The pipeline checks `store.count() == 0` before calling `get_llm()`. This ensures an empty index returns a helpful message even when Gemini is configured without an API key.

10.3 Comparison Mode Detection

File: `src/finsight/retrieval/comparison.py`

Comparison activates when:

- `compare=True` is passed explicitly, OR
- `companies` has 2+ entries, OR
- Question contains comparison keywords ("compare", "vs", "versus", "between"), AND
- Multiple companies are detected or indexed.

10.4 Financial Query Boosting

File: `src/finsight/retrieval/retriever.py`

For revenue/growth questions, a second scoped query per company is run:

```
"{company} revenue from operations growth FY 2024-25 rupee crore"
```

Why: A generic comparison question often retrieves policy/overview pages instead of financial statement tables. The boosted query pulls pages with actual numbers (e.g. IOC page 115).

10.5 Example CLI

```
python scripts/query.py "Compare revenue growth" --companies TCS,IOC -v
```

11. Prompt Engineering

File: `src/finsight/prompts/templates.py`

Two system prompts:

- `SYSTEM_PROMPT` — standard Q&A with inline citations.
- `COMPARISON_SYSTEM_PROMPT` — structured markdown sections per company + comparison summary.

Comparison rules include:

- Use `crore` columns, not `US$ millions`, for Indian reports.
- Do not invert a decline into growth.
- Quote absolute figures when unsure.

`build_rag_prompt(question, sources)` returns `(system, user)` tuple where `user` contains formatted context blocks:

```
[1] TCS | annual-report.pdf | p.58 | score=0.87
Revenue grew 4.6% in FY 2026...
```

12. API Layer

File: `api/main.py`

Endpoint	Method	Description
----------	--------	-------------

`/health`	GET	Instant health check (async, no blocking)
`/upload`	POST	Stream PDF to `/data/documents/` in 1MB chunks
`/ingest`	POST	Run ingest pipeline on uploaded file
`/query`	POST	Run query pipeline, return mode + answer
`/documents`	GET	List indexed files with chunk counts
`/documents/{name}`	DELETE	Remove all chunks for a file

Error mapping:

- 429 — Gemini daily quota exhausted
- 503 — Ollama not running / model missing
- 404 — PDF or document not found

Dependencies (api/deps.py):

- `get_store()` — singleton ChromaStore (one SQLite connection per process)

Interactive docs: <http://127.0.0.1:8000/docs>

13. Frontend (Streamlit)

File: `frontend/app.py`

Sidebar sections

- API URL — defaults to `http://127.0.0.1:8000`
- Health check — shows connected / unreachable
- Upload & ingest — PDF + metadata → API upload + ingest
- Indexed documents — list and delete
- Query mode — Auto / Single company / Compare

Chat area

Messages stored in `st.session_state.messages`. Each assistant reply shows:

- Answer text
- Mode caption (standard vs comparison)
- Expandable sources with company, file, page, score

Client: `frontend/api_client.py` — thin requests wrapper with timeouts (query: up to 600s for slow Ollama first run).

14. Integration Between Components

14.1 Three Entry Points, One Core

```
scripts/ingest.py → ingest_document()
api/main.py      → ingest_document() (via thread pool)
scripts/query.py → answer_question()
api/main.py      → answer_question() (via thread pool)
frontend/app.py  → HTTP → api/main.py
```

14.2 Provider Selection

```
EMBEDDING_PROVIDER=local → LocalEmbeddings → 384-dim vectors
EMBEDDING_PROVIDER=gemini → GeminiEmbeddings → API vectors
```

```
LLM_PROVIDER=ollama → OllamaLLM → POST localhost:11434/api/chat
LLM_PROVIDER=gemini → GeminiLLM → google-generativeai
```

Factories use `@lru_cache` — providers are singletons per process.

14.3 Chroma Metadata Filtering

Standard query with `company="TCS"`:

```
store.query(embedding, k=5, filters={"company": "TCS"})
```

Chroma applies metadata filter before similarity ranking within the filtered set.

15. Challenges and Solutions

15.1 Gemini Free-Tier Rate Limits

Problem: ~1000 embeddings/day and ~100 requests/minute on free tier. Large annual reports produce 1000+ chunks.

Solutions:

- Switched default embeddings to local BGE (unlimited).
- Batch size 10 + 70s delay when using Gemini.
- GeminiQuotaError with fast fail on daily quota (no 10-minute retry loops).
- Ingest saves after each batch; `resume=True` continues later.
- `IngestResult.partial=True` signals incomplete ingest.

15.2 API Server Hanging

Problem: Health checks timed out during long queries.

Solutions:

- Singleton ChromaStore (avoid multiple SQLite locks).
- Cached embedder/LLM (avoid reloading BGE every request).
- `run_in_threadpool` for ingest/query/document listing.
- Async /health endpoint.
- Uvicorn `--reload-dir` limited to `api`, `src`, `config` (not `.env`).

15.3 Comparison Retrieved Wrong Pages

Problem: IOC comparison returned policy pages instead of revenue tables.

Solution: Financial keyword detection + FY-scoped secondary queries with "rupee crore" phrasing.

15.4 Ollama Model Not Installed

Problem: Config said `llama3.1` but user had `llama3:latest`.

Solution: Clear error message listing installed models + ollama pull hint. Default updated in `.env.example`.

15.5 Empty Store + Missing API Key

Problem: LLM initialized before empty-store check caused errors with Gemini and no key.

Solution: Reordered `query_pipeline.py` to check `store.count()` first.

15.6 Scanned PDFs

Problem: PyMuPDF returns empty text for image-only pages.

Solution: Loader skips empty pages; ingest raises ValueError if no text extracted. OCR is out of scope for v1.

16. Best Practices Followed

- Separation of concerns — Core logic in src/finsight, HTTP in api/, UI in frontend/.
- Abstract providers — Swap embeddings/LLM via config without changing pipelines.
- Deterministic chunk IDs — Safe upsert and resume.
- Batch persistence — Never lose progress on long ingests.
- Typed settings — Pydantic catches config mistakes early.
- Testability — Fake embedders/LLMs in tests; tmp Chroma dirs.
- Singleton expensive resources — Model load once per process.
- Non-blocking health — Monitoring endpoint always responsive.
- Explicit citations — Sources flow from retrieval to UI unchanged.
- Gitignore secrets and data — .env, PDFs, Chroma files stay local.

17. Testing Strategy

56 pytest tests covering:

Area	Test file
Ingestion/cleaning/chunking	test_ingestion.py
Chroma operations	test_vectorstore.py
Gemini/local embeddings	test_embeddings.py, test_local_embeddings.py
Provider factories	test_embeddings_factory.py, test_llm_factory....
Prompts	test_prompts.py
Ingest pipeline + resume	test_ingest_pipeline.py
Query + comparison	test_query_pipeline.py, test_comparison.py
FastAPI endpoints	test_api.py

Run: .venv/bin/python -m pytest

Lint: .venv/bin/python -m ruff check .

18. How to Run the Full System

18.1 One-Time Setup

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
ollama pull llama3:latest
```

18.2 Terminal 1 — API

```
source .venv/bin/activate
uvicorn api.main:app --reload --host 127.0.0.1 --port 8000 \
--reload-dir api --reload-dir src --reload-dir config
```

18.3 Terminal 2 — Frontend

```
source .venv/bin/activate
streamlit run frontend/app.py
```

18.4 Ingest + Query

```
python scripts/ingest.py path/to/report.pdf \
--company TCS --doc-type annual_report --year 2026 --replace

python scripts/query.py "What risks did TCS mention?" --company TCS -v
```

19. Rebuilding From Scratch (Checklist)

Use this checklist to recreate FinSight independently:

- Create venv, requirements.txt, config/settings.py with Pydantic.
- Implement models.py (Chunk, Source, QueryResponse).
- Build ingestion: loader → cleaner → chunker.
- Add EmbeddingProvider + local BGE implementation.
- Wrap ChromaDB in ChromaStore with cosine space.
- Build ingest_document() with batching + resume.
- Add LLM providers (Ollama first).
- Write RAG prompt templates with citation rules.
- Implement retrieve() and answer_question().
- Add comparison detection + retrieve_per_company().
- Expose FastAPI routes; use thread pool for blocking work.
- Build Streamlit UI with API client.
- Add pytest tests with fakes for embedder/LLM.
- Configure Ruff, .gitignore, .env.example, README.

20. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation
Chunk	A slice of document text with metadata
Embedding	Numeric vector representing text meaning
Top-k	Number of nearest chunks retrieved
Cosine similarity	Measure of vector closeness (1 = identical di...
Upsert	Insert or update if id already exists
FY	Financial year (India: e.g. FY 2024-25)
ASGI	Async server interface used by FastAPI
Provider	Pluggable backend (embeddings or LLM)

21. Further Learning

To go deeper after mastering FinSight:

- Hybrid search — combine BM25 keyword search with vector search.
- Reranking — cross-encoder model to re-score top-k results.
- Evaluation — Ragas or custom metrics for answer faithfulness.

- OCR — Tesseract or cloud OCR for scanned PDFs.
- Deployment — Docker, Render, or Streamlit Cloud with secrets management.

22. Appendix A — Environment Variable Reference

Variable	Default	Description
GEMINI_API_KEY	(empty)	Required when using Gemini providers
LLM_PROVIDER	ollama	`ollama` or `gemini`
LLM_MODEL	gemini-2.0-flash	Gemini chat model name
OLLAMA_MODEL	llama3:latest	Model tag in Ollama
OLLAMA_BASE_URL	http://localhost:11434	Ollama HTTP API base
EMBEDDING_PROVIDER	local	`local` or `gemini`
EMBEDDING_MODEL	gemini-embedding-001	Gemini embedding model
LOCAL_EMBEDDING_MODEL	BAAI/bge-small-en-v1.5	sentence-transformers model
LOCAL_EMBEDDING_BATCH_SIZE	64	Batch size for local encoding
CHROMA_DIR	./storage/chroma	Chroma persistence directory
CHROMA_COLLECTION	finsight_docs	Collection name
DOCUMENTS_DIR	./data/documents	Uploaded PDF directory
CHUNK_SIZE	1000	Characters per chunk
CHUNK_OVERLAP	150	Overlap between consecutive chunks
RETRIEVAL_TOP_K	5	Chunks retrieved in standard mode
COMPARISON_K_PER_COMPANY	4	Chunks per company in comparison mode
EMBEDDING_BATCH_SIZE	10	Gemini embed batch size
EMBEDDING_BATCH_DELAY	70	Seconds between Gemini batches
FINSIGHT_API_URL	http://127.0.0.1:8000	Streamlit -> API URL

23. Appendix B — Code Walkthrough: Chunk ID Generation

File: src/finsight/ingestion/chunker.py

```
def _make_chunk_id(source_file: str, page: int, index: int, text: str) -> str:
    payload = f"{source_file}|{page}|{index}|{text}".encode()
    return hashlib.sha1(payload).hexdigest()[:16]
```

Why this design:

- Deterministic — same PDF content always yields the same ids.
- Upsert-safe — Chroma uses chunk_id as the record id; re-ingest updates instead of duplicating.
- Resume-safe — ingest can skip ids already present in existing_chunk_ids().

Example: Page 58 of annual-report-2025-2026.pdf might produce ids like a3f9c2b1e8d7041f for each sub-chunk on that page.

24. Appendix C — Code Walkthrough: ChromaStore Query

File: src/finsight/vectorstore/chroma_store.py

When you call store.query(embedding, k=5, filters={"company": "TCS"}):

- Chroma runs approximate nearest neighbor search in cosine space.
- Only records matching company=TCS are considered.
- Returns documents, metadatas, and distances.

- FinSight converts distance to similarity: $\text{score} = 1.0 - \text{distance}$.

Why one collection instead of many:

- Cross-company queries need a single search space.
- Metadata filters are fast enough for 2-5 company portfolios.
- Simpler backup and deployment (one folder: storage/chroma/).

25. Appendix D — API Request/Response Examples

Upload

```
curl -F "file=@report.pdf" http://127.0.0.1:8000/upload
```

Response:

```
{"source_file": "report.pdf", "path": ".../data/documents/report.pdf", "size_bytes": 30409432}
```

Ingest

```
curl -X POST http://127.0.0.1:8000/ingest \
-H "Content-Type: application/json" \
-d '{"source_file": "report.pdf", "company": "TCS", "doc_type": "annual_report", "year": 2026, "replace_existing": true}'
```

Query (single company)

```
curl -X POST http://127.0.0.1:8000/query \
-H "Content-Type: application/json" \
-d '{"question": "What risks did TCS mention?", "company": "TCS"}'
```

Query (comparison)

```
curl -X POST http://127.0.0.1:8000/query \
-H "Content-Type: application/json" \
-d '{"question": "Compare revenue growth", "companies": ["TCS", "IOC"], "compare": true}'
```

Response shape:

```
{
  "mode": "comparison (TCS, IOC)",
  "answer": "## TCS\n...\n\n## IOC\n...\n\n## Comparison summary\n...",
  "sources": [
    {"text": "...", "company": "TCS", "page": 58, "source_file": "...", "score": 0.87}
  ]
}
```

26. Appendix E — Ingestion Module Details

Loader (`ingestion/loader.py`)

Uses PyMuPDF (fitz) to open the PDF and iterate pages:

```
for page in doc:
    text = page.get_text("text")
    if text.strip():
        pages.append((page.number + 1, text)) # 1-indexed
```

Limitation: Image-only (scanned) pages return empty text. No OCR in v1.

Cleaner (`ingestion/cleaner.py`)

Annual reports repeat headers ("TCS Limited", page numbers) on every page. These add noise to embeddings because they appear in every chunk.

The cleaner:

- Finds lines that appear on most pages (boilerplate).
- Strips those lines from each page's text.
- Removes isolated page-number lines.

Result: Chunks contain mostly substantive content (financial tables, risk sections, MD&A).

Chunker (`ingestion/chunker.py`)

Uses separator priority: paragraph break, line break, sentence, word, then hard cut.

With `chunk_size=1000` and `overlap=150`, a 5000-character page might produce 5-6 overlapping chunks ensuring context is not lost at boundaries.

27. Appendix F — LLM Provider Details

Ollama (`llm/ollama.py`)

Calls POST `{base_url}/api/chat` with:

```
{
  "model": "llama3:latest",
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."}
  ],
  "stream": false
}
```

Before generating, `_ensure_model_available()` lists local models via Ollama API. If missing, raises a clear error with ollama pull instructions.

Timeout: 300 seconds — first inference loads the model into RAM and can take 1-2 minutes.

Gemini (`llm/gemini.py`)

Uses `google.generativeai` with retry on rate limits. Daily quota exhaustion raises `GeminiQuotaError` immediately (no long retry loops).

28. Appendix G — Frontend Session Flow

```
User opens Streamlit
-> _init_state() sets api_url, messages=[]
-> sidebar: client.health()
    OK -> "API connected"
    FAIL -> show uvicorn start instructions
-> sidebar: client.list_documents()
-> User uploads PDF + clicks "Upload & ingest"
    -> upload_pdf(name, file_stream)
    -> ingest({source_file, company, doc_type, year, ...})
    -> sidebar status: complete or partial
-> User types question in chat
    -> query({question, company/companies/compare})
```


-> append assistant message + sources to session
-> st.rerun() refreshes UI

Query modes in UI:

Mode	API payload effect
Auto	No filter; pipeline auto-detects comparison
Single company	<code>`company="TCS"`</code> filter
Compare	<code>`companies=["TCS","IOC"]`</code> , <code>`compare=true`</code>

29. Appendix H — Decision Log

Decision	Alternatives considered	Why FinSight chose this
ChromaDB	Pinecone, Weaviate, FAISS	Local, persistent, zero cost, simple Python A...
BGE-small	OpenAI embeddings, Gemini default	Free, offline, 384-dim is fast
Single collection	Per-company collections	Easier comparison queries
LangChain splitter	Manual splitting	Proven separator hierarchy
FastAPI + Streamlit	Single Gradio app	Clean API/ UI separation for portfolio
Ollama default	Gemini-only	No API key barrier for learners
SHA-1 chunk ids	UUID random ids	Deterministic resume/upsert
Thread pool in API	Celery workers	Simpler for MVP; no Redis needed

30. Appendix I — Troubleshooting Guide

Symptom	Likely cause	Fix
API unreachable in Streamlit	Uvicorn not running	Start API on port 8000
Health timeout	Server hung or reloading	Kill port 8000, restart without <code>.venv reload</code>
Ollama 503 error	Ollama not running	<code>`ollama serve`</code> or open Ollama app
Model not found	Wrong model tag	<code>`ollama pull llama3:latest`</code>
Empty ingest	Scanned PDF	Use text-based PDF or add OCR
Comparison needs 2 companies	Only one indexed	Ingest second company PDF
Partial ingest	Gemini rate limit	Re-run ingest without <code>`--replace`</code>
Wrong revenue figures	US\$ vs Rs. column	Comparison prompt rules; check source page
Slow first query	BGE model loading	Normal; cached after first call

End of FinSight Learning Guide — generated from project source for educational purposes.